BHARAT RATNA SARDAR VALLABH BHAI PATEL
RAJKIYA ENGINEERING COLLEGE, BASTI

Department of Electronics Engineering

BCC351 — Mini Project

Mini Project Report

"Arduino Based ECG Heartbeat Monitoring and Healthcare System"

A minor project report submitted in partial fulfilment of the requirement
for the award of the degree of
BACHELOR OF TECHNOLOGY
in
ELECTRONICS ENGINEERING

Submitted by

AMAN SHUKLA (Roll No. 2412130309001)
GOVIND SHUKLA (Roll No. 2412130309003)

Under the Supervision of

Ms. Garima Jaiswal
Assistant Professor, Department of Electronics Engineering
RAJKIYA ENGINEERING COLLEGE, BASTI
Basti, Uttar Pradesh – 272002

Certificate

This is to certify that AMAN SHUKLA and GOVIND SHUKLA, students of B.Tech (Electronics Engineering), have carried out the project work presented in this report entitled "Arduino Based ECG Heartbeat Monitoring and Healthcare System" in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Electronics Engineering from RAJKIYA ENGINEERING COLLEGE, BASTI, under my supervision and guidance.

The work embodies the results of the candidates' own study and effort. This report, or any part of it, has not been submitted elsewhere for the award of any other degree or diploma.

Date:
Place: Basti

Ms. Garima Jaiswal
Assistant Professor
Department of Electronics Engineering
RAJKIYA ENGINEERING COLLEGE, BASTI

Candidate's Declaration

We declare that this written submission represents our own work and ideas in our own words, and where others' ideas or words have been included, we have adequately cited and referenced the original sources. We further declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented, fabricated, or falsified any idea, data, fact, or source in this submission.

We understand that any violation of the above will be cause for disciplinary action by the University and can also evoke penal action from sources which have not been properly cited, or from whom proper permission has not been taken when needed.

AMAN SHUKLA
Roll No. 2412130309001, B.Tech (Electronics Engineering)

GOVIND SHUKLA
Roll No. 2412130309003, B.Tech (Electronics Engineering)

Approval Sheet

This project report entitled “Arduino Based ECG Heartbeat Monitoring and Healthcare System”, submitted by AMAN SHUKLA (2412130309001) and GOVIND SHUKLA (2412130309003), is approved for the award of the degree of Bachelor of Technology in Electronics Engineering.

Examiner Supervisor

Ms. Garima Jaiswal, Assistant Professor

Head of Department

Date:
Place: Basti

Acknowledgement

We express our sincere gratitude to our project guide, Ms. Garima Jaiswal, Assistant Professor, Department of Electronics Engineering, for her valuable guidance, continuous encouragement, and constructive suggestions throughout the course of this project. Her feedback was instrumental in helping us understand the practical and theoretical aspects of ECG signal acquisition and embedded systems design.

We are also thankful to the faculty members of the Department of Electronics Engineering, RAJKIYA ENGINEERING COLLEGE, BASTI, for providing the laboratory facilities and academic environment necessary to complete this mini project.

Finally, we would like to thank our family and friends for their support and encouragement during the course of this work.

AMAN SHUKLA
Roll No. 2412130309001

GOVIND SHUKLA
Roll No. 2412130309003

Abstract

This report presents the design and implementation of an Arduino-based ECG (Electrocardiogram) heartbeat monitoring prototype developed as a third-semester B.Tech mini project. The system uses an AD8232 single-lead ECG sensor module to acquire the electrical activity of the heart through three surface electrodes, and an Arduino UNO to sample the resulting analog signal and monitor the sensor's electrode lead-off status.

The Arduino continuously reads the analog ECG output on pin A0 and checks the AD8232 lead-off detection pins (LO+ and LO-) to determine whether the electrodes are making proper contact. When the electrodes are correctly connected, a green LED is lit and the raw ECG sample is transmitted over a USB serial link at 9600 baud; when a lead-off condition is detected, a red LED and a buzzer are activated to alert the user, and no ECG data is streamed. On the computer side, a Processing sketch reads the incoming serial stream and renders the ECG waveform in real time on a scrolling oscilloscope-style display.

The project demonstrates the feasibility of building a low-cost, open-hardware ECG signal acquisition and electrode-contact monitoring prototype using widely available components. The prototype does not perform heart-rate computation, arrhythmia detection, or any form of clinical diagnosis, and is intended purely as an educational demonstration of biosignal acquisition, embedded I/O handling, and serial-based visualization.

The design, implementation, and results discussed in this report are based on the actual hardware built and the firmware and visualization source code maintained in the project's public repository.

Table of Contents

Certificate i
Candidate's Declaration ii
Approval Sheet iii
Acknowledgement iv
Abstract v
List of Figures vii
List of Tables viii
List of Abbreviations ix
Chapter 1: Introduction 1
Chapter 2: Objectives 3
Chapter 3: System Design 4
3.1 Block Diagram 4
3.2 Circuit Diagram 4
Chapter 4: Background and Related Concepts 6
Chapter 5: Hardware and Software Implementation 8
5.1 Hardware Used 8
5.2 Circuit Assembly and Simulation 8
5.3 Pin Configuration 9
5.4 Firmware (Arduino Sketch) 10
5.5 Host-Side Visualization (Processing) 11
5.6 Working / Methodology 13
Chapter 6: Results and Observations 14
Chapter 7: Limitations and Future Scope 16
Chapter 8: Conclusion 17
References 18
Appendix A: Arduino Firmware Source Code 19
Appendix B: Processing Visualization Source Code 20

List of Figures

Fig. 3.1: Functional block diagram of the implemented system. 4

Fig. 3.2: Circuit schematic of the Arduino UNO – AD8232 ECG sensor interface, drawn from the verified firmware pin mapping. 5

Fig. 5.1: Cirkit Designer simulation of the Arduino UNO – AD8232 circuit used to verify the wiring before hardware assembly. 9

Fig. 5.2: Assembled hardware prototype: Arduino UNO, AD8232 ECG sensor module, breadboard wiring, status LEDs, and buzzer. 9

Fig. 6.1: Electrode placement during testing. In the AD8232's standard cable colour coding, the red lead corresponds to RA, the yellow lead to LA, and the green lead to RL; the photograph documents on-body placement during a test session. 14

Fig. 6.2: ECG waveform captured on the Arduino Serial Plotter, showing repeated sharp peaks corresponding to successive heartbeats. 15

List of Tables

Table 5.1: Hardware components used in the prototype. 8
Table 5.2: Arduino UNO to AD8232 pin mapping. 10
Table 5.3: Local indication pin mapping. 10

List of Abbreviations

Abbreviation	Full Form
ECG	Electrocardiogram
ADC	Analog-to-Digital Converter
LED	Light Emitting Diode
IDE	Integrated Development Environment
USB	Universal Serial Bus
I/O	Input / Output
PC	Personal Computer
LO+ / LO-	Lead-Off Detection (positive / negative), AD8232 output pins
GND	Ground
RA / LA / RL	Right Arm / Left Arm / Right Leg electrode positions

Chapter 1: Introduction

Cardiovascular health monitoring is one of the most widely studied areas of biomedical instrumentation, and the Electrocardiogram (ECG) remains the most fundamental non-invasive tool for observing the electrical activity of the heart. Conventional clinical ECG machines, however, are expensive, bulky, and require trained personnel to operate, which limits their use to hospitals and diagnostic centres. This has motivated a large body of hobbyist and academic work exploring low-cost, microcontroller-based ECG front ends for educational and experimental purposes.

This report documents a third-semester B.Tech mini project that implements a simple ECG signal acquisition prototype using an Arduino UNO microcontroller and an AD8232 single-lead ECG sensor module. The prototype reads the analog ECG waveform produced by the AD8232 through three adhesive body electrodes, monitors the sensor's built-in electrode lead-off detection outputs, and streams the acquired samples to a computer over a USB serial connection. A Processing sketch running on the computer plots the incoming values in real time, allowing the ECG waveform to be observed on screen.

The system additionally provides local visual and audible feedback: a green LED indicates that the electrodes are properly connected and data is being streamed, while a red LED and a buzzer activate when the AD8232 detects a lead-off condition (i.e., poor or lost electrode contact). This kind of contact-monitoring feedback is a standard and practically useful feature of AD8232-based designs, since ECG acquisition is highly sensitive to electrode placement and skin contact quality.

It is important to state clearly, at the outset, the scope of this project. The prototype described here performs signal acquisition, lead-off monitoring, and waveform visualization only. It does not calculate heart rate, does not perform any filtering or arrhythmia analysis beyond what the AD8232 analog front end itself provides, and is not a certified or clinically validated medical device. The project's purpose is educational: to demonstrate the practical interfacing of a biopotential sensor with a microcontroller, and the basic principles of serial-based real-time data visualization.

The remainder of this report is organized as follows. Chapter 2 states the objectives of the project. Chapter 3 describes the overall system design, including the functional block diagram and circuit schematic. Chapter 4 provides background on ECG signals and the technologies used. Chapter 5 describes the hardware and software implementation in detail, including the actual firmware and visualization source code. Chapter 6 presents the results and observations obtained from the working prototype. Chapter 7 discusses the limitations

of the current implementation and possible future improvements, and Chapter 8 concludes the report.

Chapter 2: Objectives

The objectives of this mini project are as follows:

- To interface an AD8232 single-lead ECG sensor module with an Arduino UNO microcontroller.
- To acquire the analog ECG signal produced by the AD8232 through the Arduino's analog-to-digital converter (pin A0).
- To monitor the AD8232's lead-off detection outputs (LO+ and LO-) so that poor or lost electrode contact can be detected.
- To transmit the acquired ECG samples to a computer over a USB serial connection at a fixed baud rate.
- To visualize the received ECG waveform in real time using a Processing sketch running on the host computer.
- To provide local visual (LED) and audible (buzzer) indication of the electrode connection status.
- To document the hardware connections, firmware, and visualization software in a form suitable for future extension.

These objectives are deliberately scoped to match the level of a third-semester undergraduate mini project: the emphasis is on correct sensor interfacing, reliable digital I/O handling, and real-time visualization, rather than on advanced signal processing or diagnostic functionality.

Chapter 3: System Design

3.1 Block Diagram

Figure 3.1 shows the functional block diagram of the implemented system. ECG electrodes placed at the right arm (RA), left arm (LA), and right leg (RL) positions pick up the heart's electrical activity. The AD8232 module amplifies and filters this weak bio-potential signal and also monitors the electrode contact quality through its LO+ and LO- outputs. The Arduino UNO samples the conditioned analog signal on pin A0 and reads the two lead-off status pins on D10 and D11. Depending on the lead-off status, the Arduino either streams the ECG sample over the serial port (electrodes connected) or withholds data and raises a local alert through the green/red LEDs and buzzer (lead-off condition). A Processing sketch on the host PC reads the incoming serial stream and renders the ECG waveform on screen.

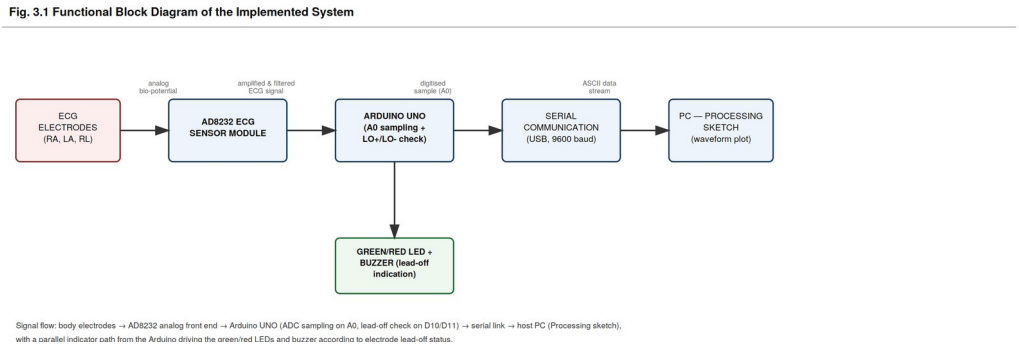


Fig. 3.1: Functional block diagram of the implemented system.

3.2 Circuit Diagram

Figure 3.2 shows the circuit schematic of the prototype, redrawn from the verified project connections (Arduino firmware pin assignments and the Cirkit Designer simulation shown later in Figure 5.1). The AD8232 module is powered from the Arduino's 3.3V and GND pins. Its analog OUTPUT is connected to Arduino analog pin A0, and its LO- and LO+ lead-off outputs are connected to Arduino digital pins D11 and D10 respectively. The green status LED is driven from digital pin D6 through a 330 Ω series resistor, the red status LED from digital pin D7 through a 330 Ω series resistor, and the buzzer directly from digital pin D8. All return paths are tied to a common ground.

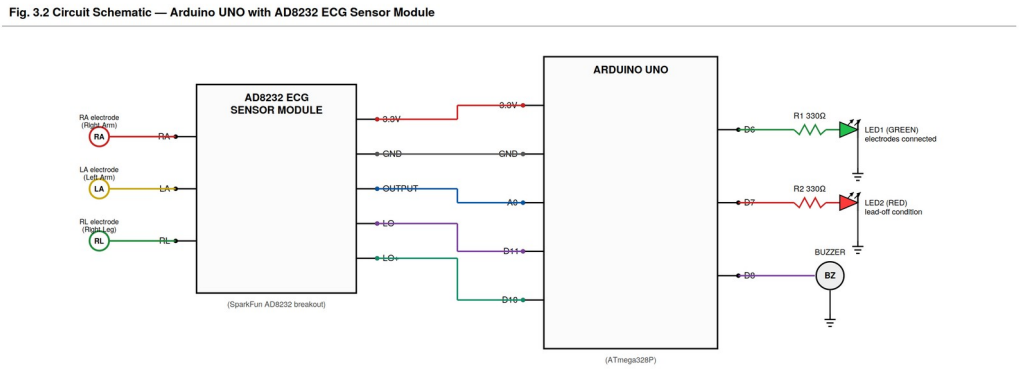


Fig. 3.2: Circuit schematic of the Arduino UNO – AD8232 ECG sensor interface, drawn from the verified firmware pin mapping.

This pin mapping matches exactly the constants declared at the top of the project's Arduino firmware (Section 5.3), and is therefore consistent between the schematic, the simulation, the physical hardware, and the source code.

Chapter 4: Background and Related Concepts

4.1 The ECG Signal

The electrocardiogram is a recording of the small electrical potentials (on the order of a few millivolts) generated by the heart muscle during each cardiac cycle. A typical ECG waveform consists of a P wave (atrial depolarization), a QRS complex (ventricular depolarization), and a T wave (ventricular repolarization). Because these signals are very small and easily corrupted by motion artefacts, mains interference, and poor electrode contact, practical ECG front-end circuits must provide amplification, filtering, and some means of detecting when an electrode has become disconnected.

4.2 The AD8232 ECG Sensor Module

The AD8232, manufactured by Analog Devices, is a single-lead, low-power integrated analog front end intended for ECG and other biopotential measurements. The breakout board used in this project (a SparkFun-style AD8232 module) exposes a 3-pin electrode connector (RA, LA, RL), an amplified analog OUTPUT pin, and two digital lead-off detection outputs, LO+ and LO-, which go high when the corresponding electrode loses contact with the skin. The module operates from a 3.3V supply and requires no external filtering components for basic operation, which makes it well suited to microcontroller-based prototypes of this kind.

4.3 Arduino UNO

The Arduino UNO is an open-source microcontroller development board built around the ATmega328P. It provides 14 digital I/O pins, 6 analog input pins, a USB interface for programming and serial communication, and is programmed using the Arduino IDE in a C/C++-based sketch format. In this project, the Arduino UNO performs three roles: it digitizes the analog ECG signal from the AD8232 through its on-board 10-bit analog-to-digital converter, it reads the two digital lead-off status lines, and it drives the LED and buzzer indicators.

4.4 Serial Communication and Processing

Serial communication over USB is used to transfer the sampled ECG data from the Arduino to a host computer at a fixed baud rate of 9600 bits per second. On the computer side, Processing — an open-source programming environment based on Java — is used to read the incoming serial data and render it as a scrolling waveform, similar in spirit to the Arduino IDE's built-in Serial Plotter, but with a custom, purpose-built display.

4.5 Related Approaches

AD8232-based ECG front ends are widely documented in hobbyist and academic embedded-systems literature, typically paired with an Arduino or similar microcontroller and either the Arduino Serial Plotter or a custom Processing/Python visualization tool. Commercial ECG machines and clinical-grade wearables achieve far higher signal fidelity and are supported by regulatory certification, but are correspondingly more expensive and are not intended for open experimentation. The prototype described in this report sits at the educational end of this spectrum: it reproduces the core acquisition and lead-off monitoring functionality of an AD8232-based front end without attempting to replicate clinical-grade filtering, calibration, or diagnostic capability.

Chapter 5: Hardware and Software Implementation

5.1 Hardware Used

Table 5.1 lists the hardware components used in the prototype, consistent with the components visible in the assembled hardware (Figure 5.2) and documented in the project repository.

Table 5.1: Hardware components used in the prototype.

Component	Purpose
Arduino UNO R3 (ATmega328P)	Main microcontroller: analog-to-digital conversion, lead-off monitoring, LED/buzzer control, and serial communication.
AD8232 ECG sensor module	Analog front end for ECG signal conditioning (amplification and filtering) and electrode lead-off detection.
ECG electrodes (RA, LA, RL) with 3.5 mm lead cable	Capture the heart's electrical activity from the body surface.
Green LED (with series resistor)	Indicates that the electrodes are properly connected.
Red LED (with series resistor)	Indicates a lead-off (poor/lost electrode contact) condition.
Buzzer	Provides an audible lead-off alert.
Breadboard and jumper wires	Prototyping and circuit interconnection.
USB cable (A to B)	Firmware upload and serial data communication with the PC.

5.2 Circuit Assembly and Simulation

Before assembling the physical circuit, the design was first verified in the Cirkit Designer simulation environment, shown in Figure 5.1. The simulated layout mirrors the final physical wiring: the AD8232 module and Arduino UNO are placed on a shared breadboard, with the LEDs and buzzer wired to their respective digital pins through the breadboard's power and ground rails.

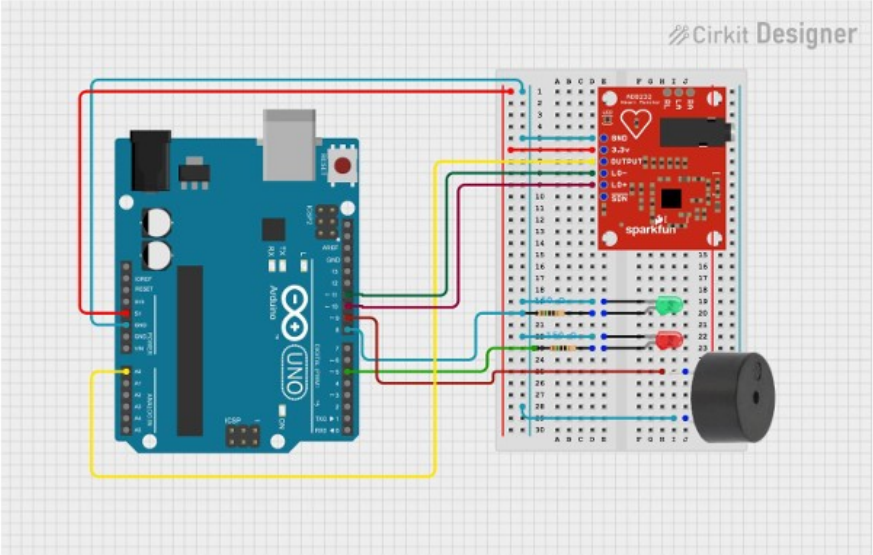


Fig. 5.1: Cirkit Designer simulation of the Arduino UNO – AD8232 circuit used to verify the wiring before hardware assembly.

Figure 5.2 shows the assembled physical hardware, consisting of the Arduino UNO, the AD8232 module mounted on a breadboard, the green and red status LEDs, the buzzer, and the three-lead electrode cable.

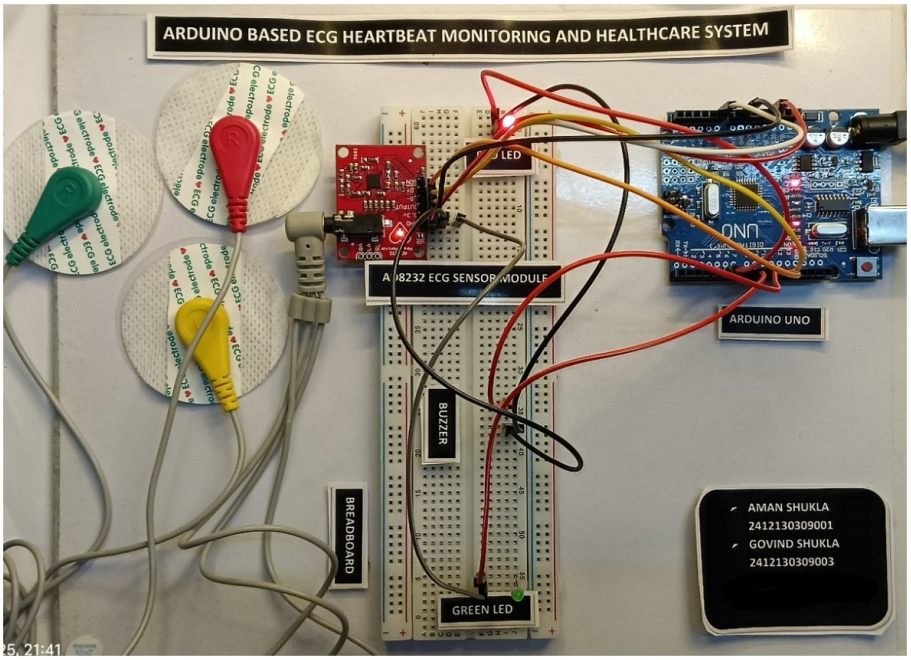


Fig. 5.2: Assembled hardware prototype: Arduino UNO, AD8232 ECG sensor module, breadboard wiring, status LEDs, and buzzer.

5.3 Pin Configuration

The pin mapping used throughout this report is taken directly from the constants declared in the project firmware (Section 5.4) and is summarized in Tables 5.2 and 5.3.

Table 5.2: Arduino UNO to AD8232 pin mapping.

Arduino UNO Pin	AD8232 Pin	Function
3.3V	3.3V	Sensor supply voltage
GND	GND	Common ground
A0	OUTPUT	Analog ECG signal input
D10	LO+	Lead-off detection (positive electrode)
D11	LO-	Lead-off detection (negative electrode)

Table 5.3: Local indication pin mapping.

Arduino UNO Pin	Component	Function
D6	Green LED (through 330 Ω resistor)	Lit when electrodes are properly connected
D7	Red LED (through 330 Ω resistor)	Lit when a lead-off condition is detected
D8	Buzzer	Sounds when a lead-off condition is detected

5.4 Firmware (Arduino Sketch)

The Arduino firmware, reproduced below and in full in Appendix A, implements the acquisition and lead-off monitoring logic described in Section 3.1. On every loop iteration, the sketch reads the analog ECG value on pin A0 and checks whether either lead-off pin (D10 or D11) is HIGH. If a lead-off condition is present, the green LED is turned off, the red LED and buzzer are turned on, and the string “Lead off” is written to the serial port instead of an ECG sample. If both leads are properly connected, the green LED is turned on, the red LED and buzzer are turned off, and the raw analog-to-digital converter reading is written to the serial port for the host computer to plot.

```
// Arduino-Based ECG Heartbeat Monitoring Prototype
// Arduino UNO + AD8232 ECG sensor

const byte ECG_PIN = A0;
const byte LEAD_OFF_PLUS = 10;
const byte LEAD_OFF_MINUS = 11;

const byte GREEN_LED = 6;
const byte RED_LED = 7;
const byte BUZZER = 8;

void setup()
{
  Serial.begin(9600);

  pinMode(LEAD_OFF_PLUS, INPUT);
  pinMode(LEAD_OFF_MINUS, INPUT);

  pinMode(GREEN_LED, OUTPUT);
  pinMode(RED_LED, OUTPUT);
```

```
  pinMode(BUZZER, OUTPUT);

  digitalWrite(GREEN_LED, LOW);
  digitalWrite(RED_LED, LOW);
  digitalWrite(BUZZER, LOW);
}

void loop()
{
  const int ecgValue = analogRead(ECG_PIN);
  const bool leadOff = (digitalRead(LEAD_OFF_PLUS) == HIGH) ||
                       (digitalRead(LEAD_OFF_MINUS) == HIGH);

  if (leadOff)
  {
    digitalWrite(GREEN_LED, LOW);
    digitalWrite(RED_LED, HIGH);
    digitalWrite(BUZZER, HIGH);

    Serial.println("Lead off");
  }
  else
  {
    digitalWrite(GREEN_LED, HIGH);
    digitalWrite(RED_LED, LOW);
    digitalWrite(BUZZER, LOW);

    // Send the ECG sample to the Arduino Serial Plotter.
    Serial.println(ecgValue);
  }

  delay(5);
}
```

Source: Arduino/ECG_Heartbeat_Monitor.ino, project repository amanshukla87/ECG-Heartbeat-Monitoring-System (see References).

5.5 Host-Side Visualization (Processing)

On the computer side, a Processing sketch opens the Arduino's serial port at 9600 baud, reads each incoming line, and appends the parsed value to a circular buffer that is redrawn every frame as a scrolling green waveform on a black, oscilloscope-style background, with a light grid and a centre reference line for readability. Only values within the valid 10-bit ADC range (0–1023) are accepted, and malformed lines are safely ignored.

```
import processing.serial.*;

Serial myPort;

int[] ecgValues;
int writeIndex = 0;

void setup() {
  size(800, 400);

  // Arduino serial port
  myPort = new Serial(this, "COM17", 9600);
```

```
  // Clear any old serial data
  myPort.clear();

  // ECG buffer
  ecgValues = new int[width];

  // Initialize buffer to center line
  for (int i = 0; i < width; i++) {
    ecgValues[i] = 512;
  }

  background(0);
  frameRate(60);
}

void draw() {
  background(0);

  // Read ECG data from Arduino
  while (myPort.available() > 0) {
    String inData = myPort.readStringUntil('\n');

    if (inData != null) {
      inData = trim(inData);

      if (inData.length() > 0) {
        try {
          int value = int(inData);

          // Accept normal Arduino ADC range
          if (value >= 0 && value <= 1023) {
            ecgValues[writeIndex] = value;
            writeIndex = (writeIndex + 1) % width;
          }
        }
        catch (Exception e) {
          // Ignore invalid serial data
        }
      }
    }
  }

  // Draw ECG grid
  stroke(40);
  strokeWeight(1);

  for (int y = 50; y < height; y += 50) {
    line(0, y, width, y);
  }

  for (int x = 0; x < width; x += 50) {
    line(x, 0, x, height);
  }

  // Center reference line
  stroke(80);
  line(0, height / 2, width, height / 2);

  // Draw ECG waveform
  stroke(0, 255, 0);
  strokeWeight(2);
  noFill();

  beginShape();
```

```
  for (int x = 0; x < width; x++) {
    int bufferIndex = (writeIndex + x) % width;
    float y = map(ecgValues[bufferIndex], 0, 1023, height - 30, 30);
    vertex(x, y);
  }

  endShape();

  // Display information
  fill(0, 255, 0);
  textSize(16);
  text("ECG Heartbeat Monitoring", 20, 25);

  fill(180);
  textSize(12);
  text("Arduino: COM17 | Baud Rate: 9600", 20, height - 10);
}
```

Source: Processing/ECG_Waveform_Processing.pde, project repository amanshukla87/ECG-Heartbeat-Monitoring-System (see References). Note that the serial port name (“COM17”) is hard-coded to the development machine used by the authors and would need to be changed to match the port assigned on a different computer.

5.6 Working / Methodology

The overall working of the system can be summarized as follows: (1) the three ECG electrodes are attached to the body at the RA, LA, and RL positions and connected to the AD8232 module through the 3.5 mm lead cable; (2) the AD8232 continuously conditions the resulting bio-potential signal and monitors electrode contact through its LO+/LO- outputs; (3) the Arduino firmware polls the lead-off pins and the analog input on every loop iteration; (4) if the electrodes are properly connected, the green LED lights up and the current ADC sample is written to the serial port; if a lead-off condition is detected, the red LED and buzzer activate instead and no ECG sample is sent; (5) on the host computer, the Processing sketch reads the incoming serial stream and plots the waveform in real time. This cycle repeats continuously (with a 5 ms delay per loop in the firmware) for as long as the Arduino remains connected and powered.

Chapter 6: Results and Observations

The assembled prototype was tested by attaching the three electrodes to a volunteer's forearms, as shown in Figure 6.1, and powering the Arduino from a USB connection to a laptop running the Arduino IDE and the Processing sketch described in Section 5.5.

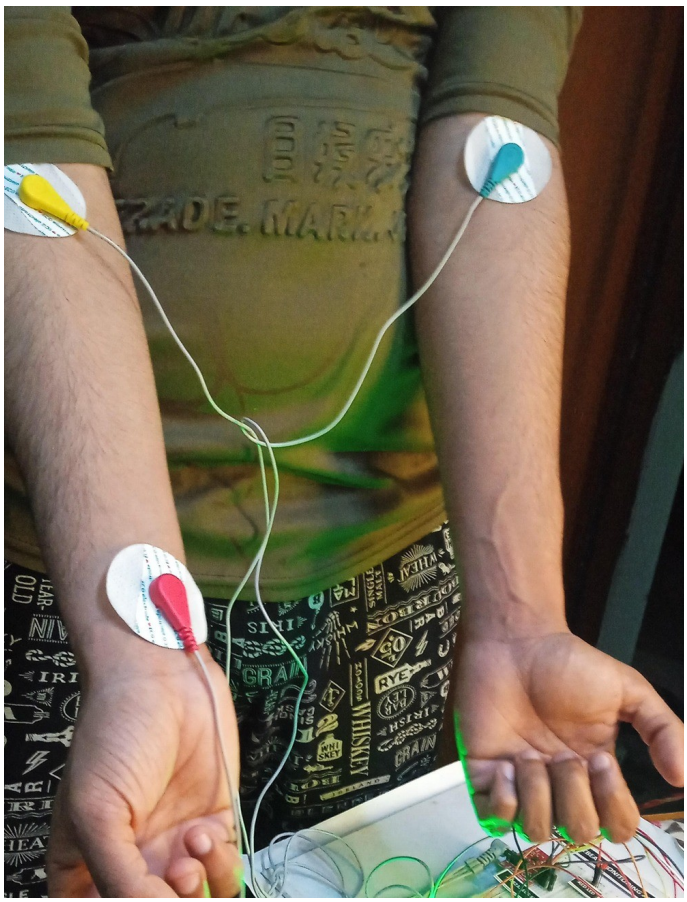


Fig. 6.1: Electrode placement during testing. In the AD8232's standard cable colour coding, the red lead corresponds to RA, the yellow lead to LA, and the green lead to RL; the photograph documents on-body placement during a test session.

With both electrodes making good skin contact, the green LED remained lit and the red LED and buzzer stayed off, confirming correct lead-off detection behaviour. The Processing sketch and the Arduino IDE's own Serial Plotter both displayed a repeating waveform with a clearly visible sharp peak corresponding to each heartbeat, separated by a comparatively flat baseline segment, consistent with the expected general shape of a single-lead ECG trace. Figure 6.2 shows a representative capture of this waveform from the Arduino Serial Plotter.

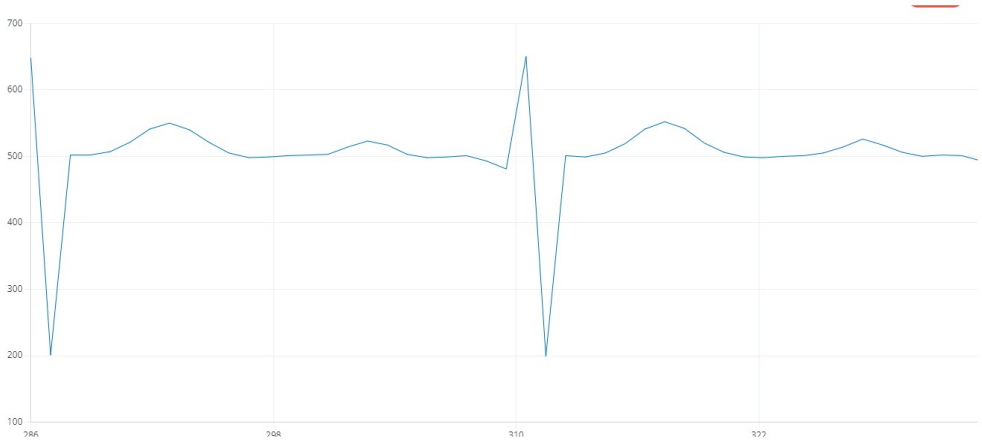


Fig. 6.2: ECG waveform captured on the Arduino Serial Plotter, showing repeated sharp peaks corresponding to successive heartbeats.

When an electrode was deliberately lifted away from the skin, the AD8232's LO+/LO- outputs went high within the firmware's polling interval, the red LED and buzzer activated immediately, the green LED turned off, and the Processing display stopped receiving new ECG samples — as expected, since the firmware withholds ECG data while a lead-off condition is present. Reattaching the electrode restored normal green-LED operation and waveform streaming within a fraction of a second.

These observations confirm that the two core functions of the prototype — ECG waveform acquisition/visualization, and electrode lead-off monitoring with local alerting — both work as designed. No quantitative heart-rate figures are reported here, since the current firmware does not perform heart-rate extraction; only the raw waveform is acquired and displayed.

Chapter 7: Limitations and Future Scope

7.1 Limitations

- The system is not a certified medical device and must not be used for clinical diagnosis; it is an educational prototype only.
- The firmware does not compute heart rate (beats per minute) or perform any peak-detection or arrhythmia analysis — it only streams the raw, lead-off-gated ADC samples.
- Signal quality is sensitive to electrode placement, motion artefacts, and ambient electrical noise, as is typical of any single-lead, un-shielded ECG front end.
- The Processing sketch's serial port name is hard-coded and must be edited manually to match the port used on a different computer.
- The prototype has no on-board data logging; ECG data is only visualized live and is not stored for later analysis.

7.2 Future Scope

Several extensions could be considered in future iterations of this project, without altering the current, verified prototype:

- Adding firmware-side peak detection to estimate heart rate in beats per minute from the acquired waveform.
- Logging ECG data to an SD card or to the host computer for offline review.
- Adding a wireless link (e.g., Bluetooth or Wi-Fi) so that the Arduino does not need to remain tethered to the host computer by USB.
- Extending the visualization to a mobile application for more convenient at-home use.
- Exploring digital filtering (e.g., a moving-average or notch filter) to further reduce motion and mains-interference artefacts.

These items are proposed as possible future work and are not part of the current implementation described in this report.

Chapter 8: Conclusion

This mini project successfully demonstrates the interfacing of an AD8232 single-lead ECG sensor module with an Arduino UNO to acquire, monitor, and visualize a real ECG waveform. The implemented firmware correctly reads the analog ECG signal on pin A0, monitors electrode contact through the AD8232's LO+/LO- lead-off outputs, and provides clear local feedback through a green LED, a red LED, and a buzzer. A companion Processing sketch on the host computer visualizes the incoming waveform in real time, and testing confirmed that both the waveform display and the lead-off alerting behave as intended.

As an undergraduate mini project, the work achieves its stated objectives: correct sensor interfacing, reliable digital I/O handling for lead-off monitoring, and real-time serial-based visualization, all built from low-cost, widely available components. The project deliberately does not claim heart-rate computation, clinical diagnostic capability, or any form of medical certification, and it is presented purely as an educational demonstration of embedded biosignal acquisition.

Overall, this project provided practical, hands-on experience with analog and digital sensor interfacing, embedded firmware development, serial communication protocols, and real-time data visualization — forming a useful foundation for further study in embedded systems and biomedical instrumentation.

References

[1] Arduino, “Arduino UNO Rev3 — Technical Specifications,” Arduino Official Documentation. Available: https://docs.arduino.cc/hardware/uno-rev3

[2] Arduino, “Arduino IDE Documentation,” Arduino Official Documentation. Available: https://docs.arduino.cc/software/ide/

[3] Analog Devices, “AD8232: Single-Lead, Heart Rate Monitor Front End — Data Sheet,” Analog Devices Inc. Available: https://www.analog.com/en/products/ad8232.html

[4] Processing Foundation, “Processing Environment Documentation,” Available: https://processing.org/environment/

[5] A. Shukla, “ECG-Heartbeat-Monitoring-System” (project source repository), GitHub. Available: https://github.com/amanshukla87/ECG-Heartbeat-Monitoring-System

Appendix A: Arduino Firmware Source Code

The complete Arduino firmware, exactly as maintained in the project repository (Arduino/ECG_Heartbeat_Monitor.ino), is reproduced below.

```
// Arduino-Based ECG Heartbeat Monitoring Prototype
// Arduino UNO + AD8232 ECG sensor

const byte ECG_PIN = A0;
const byte LEAD_OFF_PLUS = 10;
const byte LEAD_OFF_MINUS = 11;

const byte GREEN_LED = 6;
const byte RED_LED = 7;
const byte BUZZER = 8;

void setup()
{
  Serial.begin(9600);

  pinMode(LEAD_OFF_PLUS, INPUT);
  pinMode(LEAD_OFF_MINUS, INPUT);

  pinMode(GREEN_LED, OUTPUT);
  pinMode(RED_LED, OUTPUT);
  pinMode(BUZZER, OUTPUT);

  digitalWrite(GREEN_LED, LOW);
  digitalWrite(RED_LED, LOW);
  digitalWrite(BUZZER, LOW);
}

void loop()
{
  const int ecgValue = analogRead(ECG_PIN);
  const bool leadOff = (digitalRead(LEAD_OFF_PLUS) == HIGH) ||
                       (digitalRead(LEAD_OFF_MINUS) == HIGH);

  if (leadOff)
  {
    digitalWrite(GREEN_LED, LOW);
    digitalWrite(RED_LED, HIGH);
    digitalWrite(BUZZER, HIGH);

    Serial.println("Lead off");
  }
  else
  {
    digitalWrite(GREEN_LED, HIGH);
    digitalWrite(RED_LED, LOW);
    digitalWrite(BUZZER, LOW);

    // Send the ECG sample to the Arduino Serial Plotter.
    Serial.println(ecgValue);
  }

  delay(5);
}
```

Appendix B: Processing Visualization Source Code

The complete Processing sketch used for host-side ECG waveform visualization, exactly as maintained in the project repository (Processing/ECG_Waveform_Processing.pde), is reproduced below.

```
import processing.serial.*;

Serial myPort;

int[] ecgValues;
int writeIndex = 0;

void setup() {
  size(800, 400);

  // Arduino serial port
  myPort = new Serial(this, "COM17", 9600);

  // Clear any old serial data
  myPort.clear();

  // ECG buffer
  ecgValues = new int[width];

  // Initialize buffer to center line
  for (int i = 0; i < width; i++) {
    ecgValues[i] = 512;
  }

  background(0);
  frameRate(60);
}

void draw() {
  background(0);

  // Read ECG data from Arduino
  while (myPort.available() > 0) {
    String inData = myPort.readStringUntil('\n');

    if (inData != null) {
      inData = trim(inData);

      if (inData.length() > 0) {
        try {
          int value = int(inData);

          // Accept normal Arduino ADC range
          if (value >= 0 && value <= 1023) {
            ecgValues[writeIndex] = value;
            writeIndex = (writeIndex + 1) % width;
          }
        }
        catch (Exception e) {
          // Ignore invalid serial data
        }
      }
    }
  }
```

```
  // Draw ECG grid
  stroke(40);
  strokeWeight(1);

  for (int y = 50; y < height; y += 50) {
    line(0, y, width, y);
  }

  for (int x = 0; x < width; x += 50) {
    line(x, 0, x, height);
  }

  // Center reference line
  stroke(80);
  line(0, height / 2, width, height / 2);

  // Draw ECG waveform
  stroke(0, 255, 0);
  strokeWeight(2);
  noFill();

  beginShape();

  for (int x = 0; x < width; x++) {
    int bufferIndex = (writeIndex + x) % width;
    float y = map(ecgValues[bufferIndex], 0, 1023, height - 30, 30);
    vertex(x, y);
  }

  endShape();

  // Display information
  fill(0, 255, 0);
  textSize(16);
  text("ECG Heartbeat Monitoring", 20, 25);

  fill(180);
  textSize(12);
  text("Arduino: COM17 | Baud Rate: 9600", 20, height - 10);
}
```